高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



致　辞

陈敏

CANN开源社区PMC主席



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



CANNBench Season1 总结

王东旭、高雪健

cann-bench Committer



CANNBench 昇腾算子评测基准 · 阶段总结（2026.05–2026.10）

从 cann-bench 开源框架到 CANNBench 社区生态 · 六个月演进全景

核心构建目标

1. 生产级标准

算子选取 | 精度标准 | 性能采集 | 泛化性要求

2. agent-native

接口cli/mcp化 | 防作弊机制

3. 科学评价

HAP (Hardware-Anchored Performance)

2026.05 ★ 关键里程碑

框架开源起步

- cann-bench 开源仓 Release
- 提出 HAP 性能评分口径
- 支持多卡并行评测
- 支持 AscendC 算子评测

2026.06

评测能力成型

- 评测网站设计开发
- 支持 Ascend 950 硬件评测
- 初版反作弊机制
- 评测报告系统
- 评测集隐藏用例构建

2026.07 ★ 关键里程碑

网站发布 & 平台化

- CANNBench 评测网站发布（cannbench.com）
- 反作弊机制加固
- Runner 设备负载感知与动态调度
- 算子工作台与聚合方案
- MCP 服务上线

2026.08

多生态接入与稳定性

- 支持 Triton / TileLang / PyPTO / PyPTO-Pro 多种提交方式
- 评测 host 内存治理
- 团队方案横向对比
- 跨平台 OOM 治理

2026.09

第二赛季筹备

- Season2 评测集设计与内测
- 版本迁移功能
- 新增 Ascend 950DT 榜单
- 开源仓易用性整改
- 后台管理优化

2026.10

社区运营

- CANNBench Season1 线下技术沙龙

★ 关键里程碑：5 月 cann-bench 开源仓 Release | 7 月 CANNBench 评测网站发布（cannbench.com） · HAP = Hardware-Anchored Performance

运营情况总览：流量、提交与稳定性

统计窗口：网站上线至 2026.10.16 · 数据来源：CANNBench 运营后台

756
用户数
网站上线以来累计

5,628
独立访客
注册 716 · 游客 4,804

237,429
页面浏览 PV
最大来源 gitcode.com（44,088 次）

30,229
提交数
用户提交 26,313 · 迁移任务 351 · Debug 21

任务耗时与提交平台分布

11m42s
平均任务耗时

1m5s
平均编译耗时

提交平台分布



最大访问来源：**gitcode.com**，带来 44,088 次访问

MCP 接入

306,295
API 调用（次）

158
活跃接入方

99.00%
请求成功率

99.982%
接入可用度

MCP 服务于 7 月上线，10 月随 v1.1.2 发布 MCP 0.9.1，支撑 AI Agent 自动化打榜接入。

稳定性与服务可用度

96.84% 平台交付率
36,923 / 38,126 次执行产出可结算结果

93.36% 有效执行成功率
完全成功 24,100 + 部分成功 10,371

硬失败率 **3.16%** | 网站探针可用度 **94.672%**

P95 排队等待 **47 min**（P50 仅 1 s）

P95 评测耗时 **40 min**（P50 3 min）

日均交付吞吐 **423.6** 次执行/天

Runner 算力 **34 台**（A3 10 · 950PR 15 · 950DT 9）

注：PV / 提交 / 交付等均为累计口径；交付率 = 产出可结算结果的执行占比；P50 / P95 为任务排队与评测耗时分位数。

排行榜与打榜情况：三大硬件榜单

评测集 official-tasks v1.1 · 53 算子 · 数据截至 2026.10.15

A3 榜

170 方案 · 竞争最激烈

1	ForgeKernel 加速比 1.835× · 通过率 99.0% · 10.15 更新	8,566.11
2	infynai-kernaut 加速比 1.474× · 通过率 99.3%	8,186.17
3	SLAI-CUHKSZ-KernelCAT 加速比 1.424× · 通过率 99.5%	8,162.86

整体榜单情况

打榜方案数 170 | 全榜测例通过率 52.1%

中位基准加速比 0.756×（理论 0.07×）

榜首领先第 2 名 379.94 分 · 前三名分差 <5%，竞争激烈

950PR 榜

114 个方案打榜

1	CANNBot EasyAsc 450USD budget 加速比 1.659× · 通过率 99.3% · 10.15 更新	8,591.76
2	CANNBot_CAKE 加速比 1.334× · 通过率 98.6%	8,335.17
3	CANNBOT PYPTO 加速比 1.209× · 通过率 97.7%	8,031.53

整体榜单情况

打榜方案数 114 | 全榜测例通过率 46.8%

中位基准加速比 0.536×（理论 0.09×）

榜首领先第 2 名 256.59 分

950DT 榜

23 方案 · 9 月新增

1	CANNBOT_EasyASC_DT 加速比 0.482× · 通过率 98.6%	6,966.15
2	CANNBOT_direct_DS 加速比 0.402× · 通过率 92.3%	6,233.16
3	CANNBOT_direct_92 加速比 0.205× · 通过率 88.1%	6,123.25

整体榜单情况

打榜方案数 23 | 全榜测例通过率 72.0%（三榜最高）

中位基准加速比 0.260×（理论 0.06×）

榜首领先第 2 名 732.99 分 · 梯度明显

打榜观察

- 10 月中旬打榜进入白热化：A3 与 950PR 榜首均于 10.15 刷新纪录
- 头部方案普遍实现 **53/53 全算子覆盖**、测例通过率 **>99%**，榜首基准加速比最高 1.835×
- 独立开发者与高校 / 企业团队共同参与

CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



Evolving Context to Fit the Harness in CANN Kernel Optimization

张睿劼

深圳河套学院与智子芯源联合团队 技术专家



KernelBridge

基于上下文演化的算子内核搜索



CANN AtomGit

目录

① Context

模型、知识库与本轮上下文

归档资料，记录经验，组装本轮输入

② Search

方案选择与多步迭代

条件剪枝、代码变异与会话调度

③ Replay

轨迹复盘与上下文修订

裁剪旧内容，补充条件，继续优化

CANNBench 的任务约束与优化空间

任务约束

任务与评测规则不变，在给定预算下优化内核

任务定义与评测规则 → 共享预算 B → 通过验证的最快程序

可优化的实现

算法	计算结构	访存方式	并行划分	执行调度	硬件映射
改变计算量	分解与融合	数据复用与布局	核间任务分配	流水重叠与同步	指令与资源配置

➤ 我们固定任务与评测规则，再尝试调整算法实现、访存方式和硬件映射。

KernelBridge 方法总览



CANN AtomGit

① Context

模型、知识库与本轮上下文

查阅资料，记录经验，为本轮优化准备输入

1.1 系统组件及其更新方式

核心组件	内容与作用	更新方式
基座模型 M	提供推理和代码生成能力	全程不微调，模型参数不变
原始资料库 D	文档、代码与论文原文 保留来源和版本记录	新资料追加归档 已有版本保留，便于追溯
持久知识库 K	优化目标、改动与适用条件 区分事实、假设与观测	当前任务只读 修订通过验证后追加版本
本轮上下文 C	检索结果、父代程序 执行反馈与当前优化假设	每轮加入新反馈 复盘后重建下一轮输入



- 知识库 K 保存经验和证据，本轮上下文 C 汇集当前任务需要的信息。
- 会话中先试用修订，通过验证后再追加知识版本。

1.2 记录优化知识，组装本轮输入

查阅资料与历史记录

- 厂商文档
- 社区代码与仓库
- 论文与技术讨论
- 成功与失败记录

提取并核对

持久知识库 K

按版本保存，例如 K2@v1

目标 intent	希望减少哪项开销
条件	哪些输入与资源下适用
改动	如何改计算与数据流
开销	省下什么，又增加什么
依据	来源事实 / 待验假设 / 本次观测
引用	文档、源码版本与评测回执

按需检索

本轮上下文 C

任务要求与父代代码
检索到的知识和原文
评测反馈与当前假设

交给编码会话

➤ 知识卡写明目标、改动和依据，编码会话按当前任务取用。

1.3 优化方案与适用条件

Maximum（逐元素取最大值）：扩大单批处理行数

优化方案卡

目标 intent	减少分批调度开销
改动	扩大单批处理行数
适用条件	输入匹配、资源够用、批次能扩大
预期省下	循环与调度开销
需要增加	片上 UB 空间

条件对应

源码中的 MakePlan

核对输入形状与广播关系
按任务量和 UB 容量推算单批行数
检查批次能否超过父代

```
if (batchRows <=
    static_cast<uint64_t>(kParentBatchRows))
    return plan;
```

这段检查对应“批次能扩大”

➤ 方案卡说明怎么改、何时适用，以及预计省下和新增的开销。

CANN AtomGit

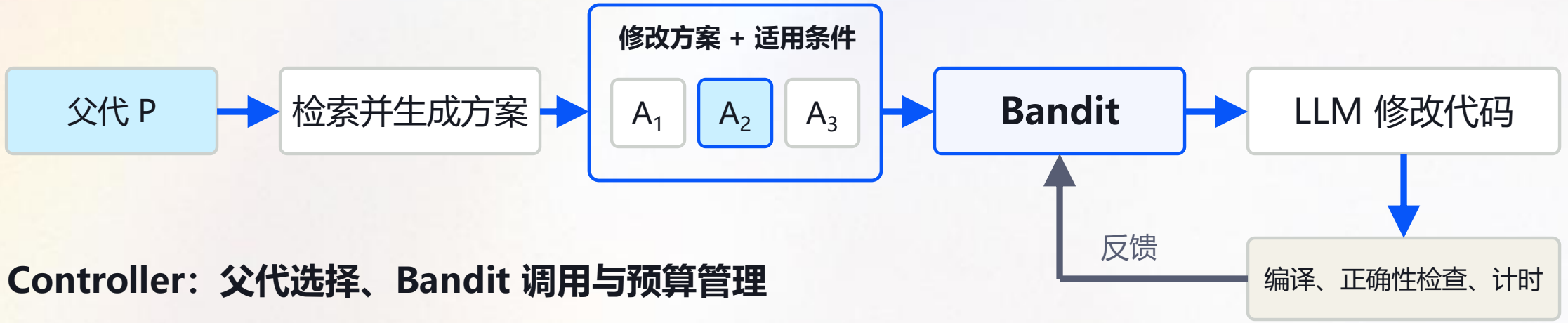
② Search

方案选择与多步迭代

条件剪枝与 Bandit 方案选择，代码变异与会话调度

2.1 选择优化方案并生成新程序

Agent Harness

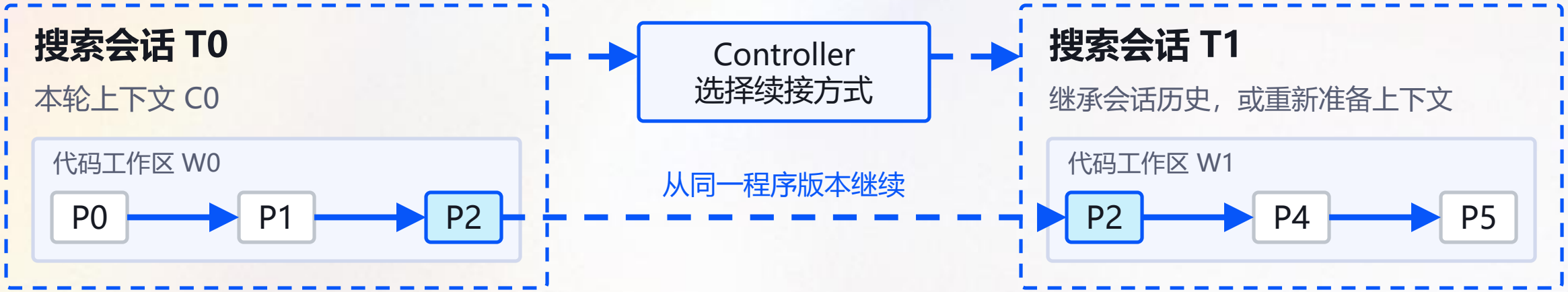


Controller：父代选择、Bandit 调用与预算管理

- Controller 调用 Bandit 选择方案、分配预算，编码会话负责改代码。
- 每次评测后，系统更新收益统计，再决定下一步试什么。

2.2 多步迭代与搜索续接

任务共享预算 B：会话切换和新建 worktree 均不重置额度



继续当前会话

方案仍在推进
按原计划继续迭代

Fork 会话

需要另试一个方向
保留已有会话历史

Replay 后新建会话

重复失败或结果不符预期
复盘后换一份上下文继续

➤ Controller 分别管理会话、代码工作区和预算，续接从已验证的程序版本出发。

2.3 改代码前先筛选方案

Maximum（逐元素取最大值）：扩大单批处理行数

筛选流程示意



➤ 搜索器先核对输入与资源，跳过已知不适用的方案，再分配编译和评测预算。

CANN AtomGit

③ Replay

轨迹复盘与上下文修订

复盘尝试结果，修正当前假设，接着优化程序

3.1 对照执行结果，修正优化假设

优化记录

修改方案与预期

父代代码、知识条目与当前假设

执行结果与证据

源码差异、正确性与性能回执

完整记录保存在任务账本中

Replay
对照预期与结果

记录上下文改动

保留 任务约束、有效证据和程序引用

裁剪 无关资料、重复输出和过时建议

修订 优化假设、适用条件和后续尝试

组装下轮输入 C1

➤ Replay 保留有用证据，裁掉无关内容，并把需要修改的假设写进下一轮输入。

3.2 复盘分桶反例，追加条件修订

Scatter（按索引写入）：补上分桶开销，修正成本估计

Replay：对照实测



scatter_14，同一用例的两次阶段实测

E1 本次观测：分桶路径更慢

复盘记录指出：漏算了固定开销

追加修订

修订草案 D1

引用 K2@v1

intent 减少重复扫描，避免小输入变慢

改动沿用：先分桶，再按桶扫描

- 成本估计：未计足分桶与启动开销

+ 条件：预计节省 > 新增成本 + 余量
+ 代价：分桶初始化、重排和启动

H1 待验假设　引用：父代与分桶回执

加载修订 → 核算方案开销 → 安排尝试与验证 → 记录新结果

➤ Replay 按知识卡的字段追加修订，写清目标、改动和证据，再交给下一轮验证。

3.3 重建上下文，接着优化

当前任务：沿用任务规则、知识库版本与累计预算

原会话 T0

本轮上下文 C0

任务、方案与尝试结果

Replay 复盘

保留有用信息，裁剪旧内容
修正假设，追加修订草案

后继会话 T1

加载下轮输入 C1

代码工作区 W1

P2 → 继续改程序

保留程序 P2
源码快照与评测回执

恢复同一源码版本

任务结束后：验证并复用

并行改代码时，各会话用独立 worktree

保留修订草案 → 在其他任务上验证 → 追加 K 新版本 → 供新任务调用

➤ 当前任务先试用新上下文，修订通过验证后再写入知识库，供新任务调用。

KernelBridge：执行反馈驱动的上下文演化



保留程序与执行证据，更新下一轮优化所需的上下文

CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



CANNBot-CAKE: Collaborative Agentic Kernel Evolution for Ascend NPU

陈薪先

CANNBot性能调优技术专家



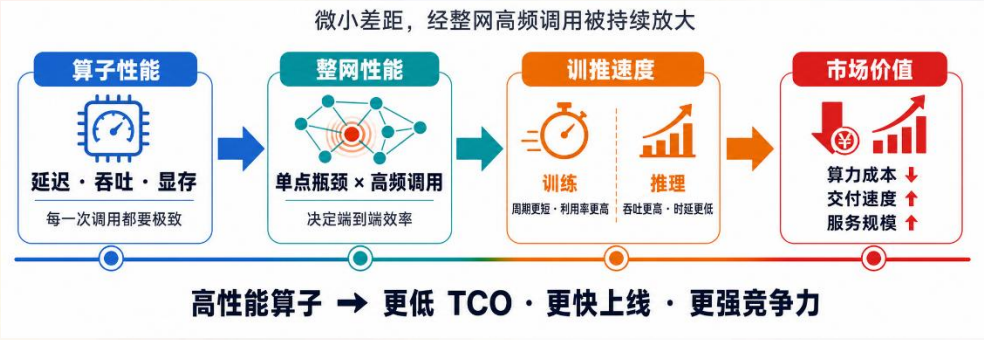
CANNBot-CAKE: Collaborative Agentic Kernel Evolution for Ascend NPU



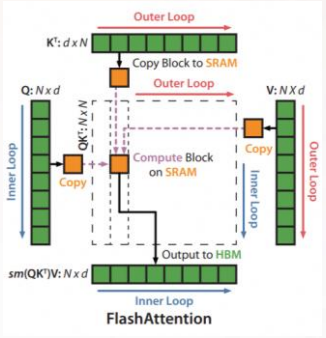
CANN AtomGit

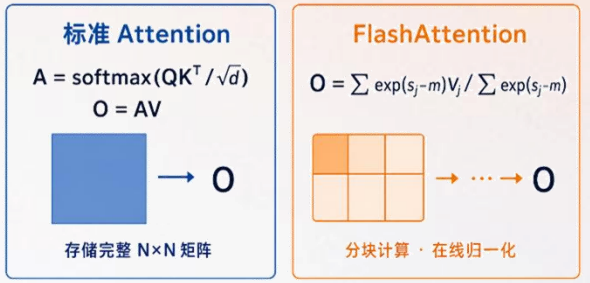
算子自动生成与优化: 挑战与方案

① 算子编程对性能要求极高



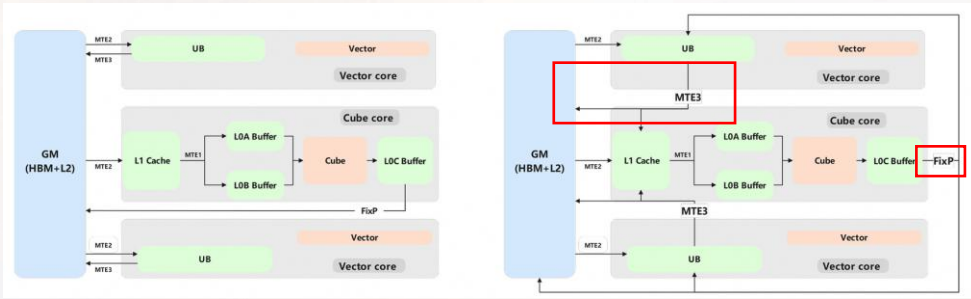
② 需要复杂的数学算法技巧





相同结果，不同算法逻辑 -- 显存读写骤减，性能翻倍

③需要熟悉各代际硬件架构与指令集



昇腾 910 系列　　昇腾 950

④ 主流模型相关语料缺乏，编写昇腾算子能力较弱

开源语料差距 → 算子能力差距



训练样本稀疏 → 模型编写昇腾算子能力较弱

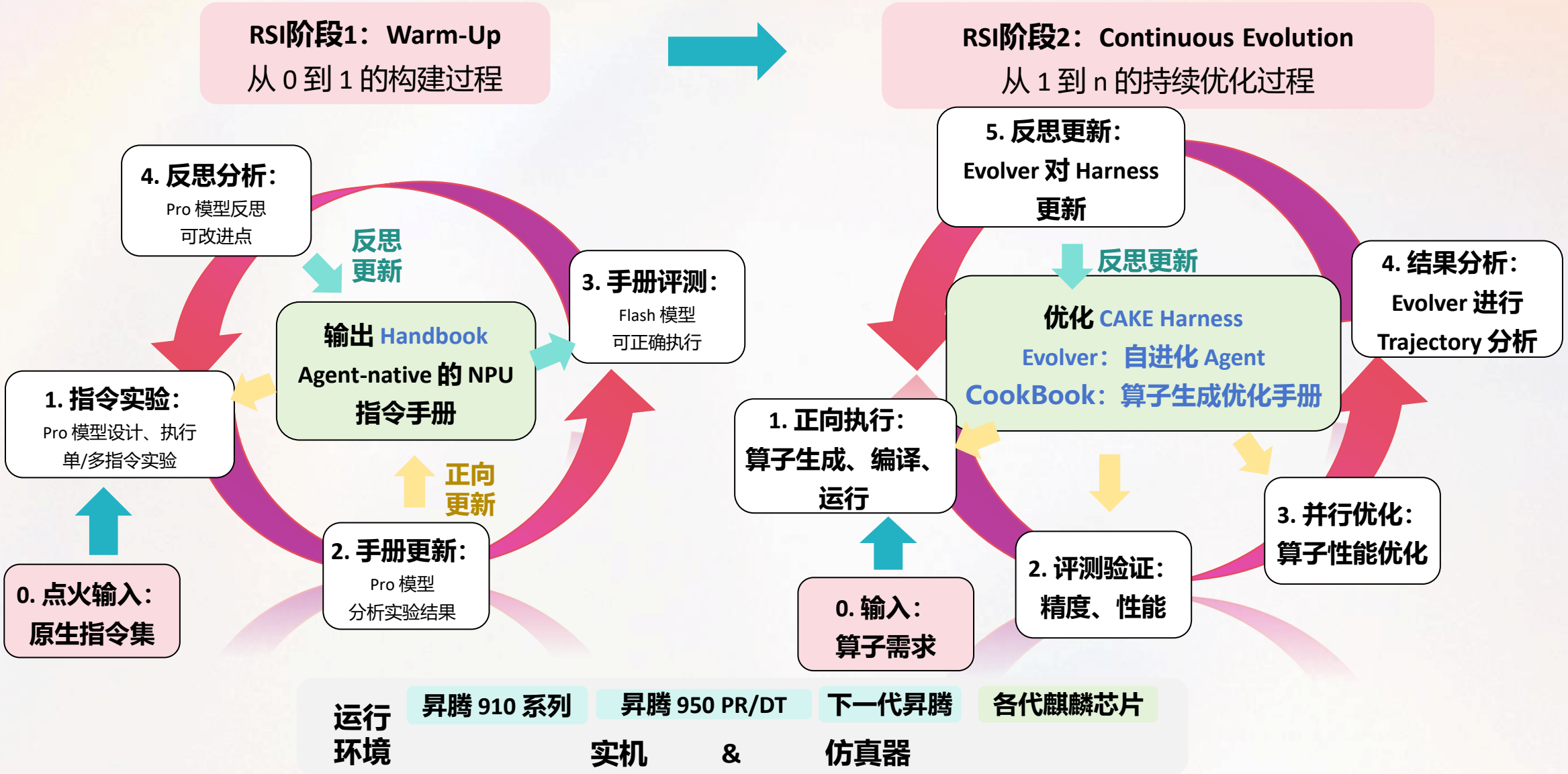
Models	CUDA (%)			AscendC (%)		
	Comp@5	Pass@5	SU_1@5	Comp@5	Pass@5	SU_1@5
DeepSeek-V3-0324	98.2	35.1	13.7	7.7	2.8	1.4
Qwen3-235B	97.2	43.9	13.0	3.9	2.1	1.4
Qwen2.5-Coder-32B	94.7	31.9	11.9	4.2	2.5	1.1
GPT-4o	**99.6**	32.3	12.6	3.2	2.1	0.7
Claude-Sonnet-4	97.9	**55.8**	**26.0**	**8.1**	**3.9**	**2.1**

arXiv:2507.17773

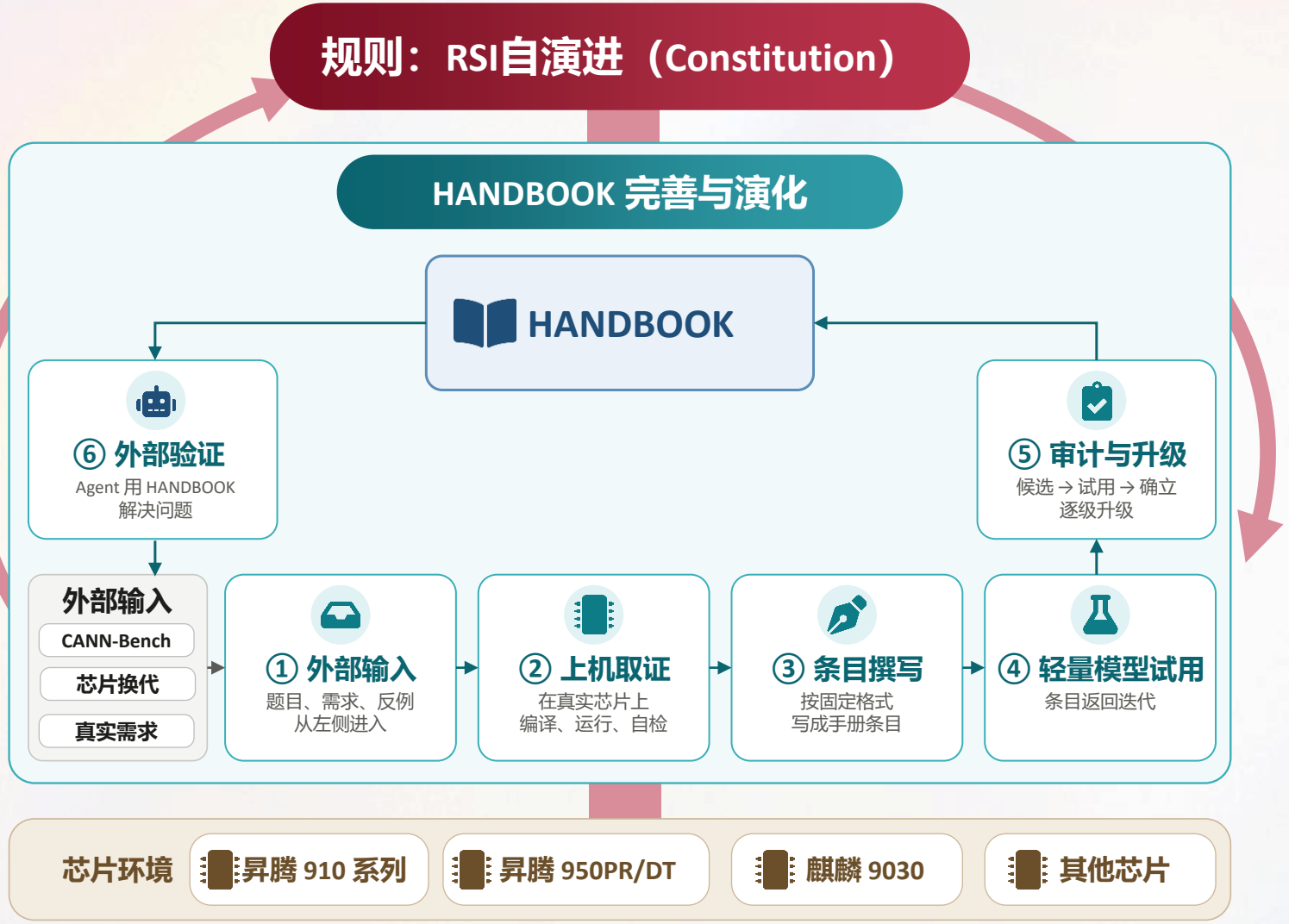
1. 构建 Agent Native 知识库：以 Agent 能力为基础，构建更方便检索的结构化、标准化的关键知识库。
2. 构建自进化 Agent 体系：Agent 生成速度超过人审核能力，通过自监督与关键知识专家微调确保 Agent 进化质量。

CANN AtomGit

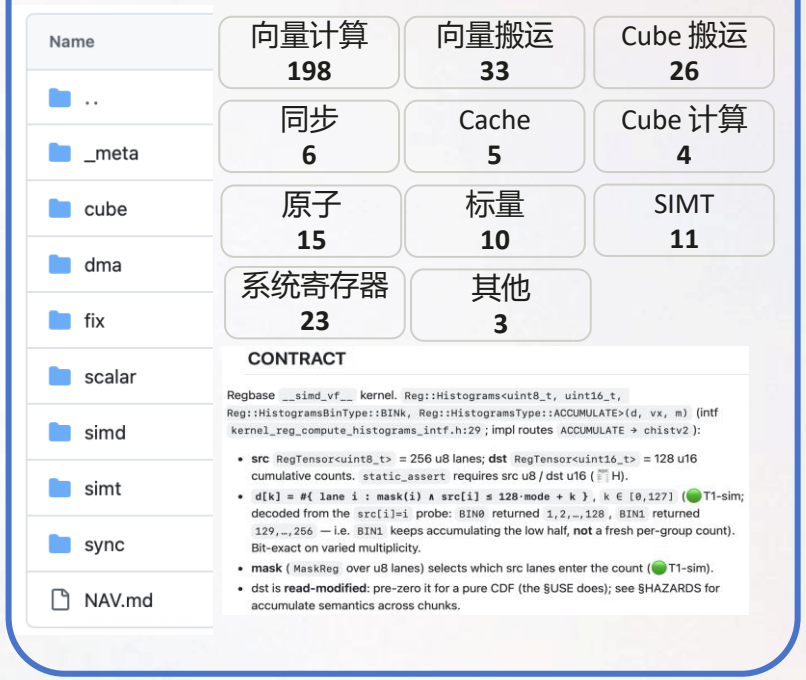
CAKE RSI: CAKE 项目基于 RSI 的自我构建过程



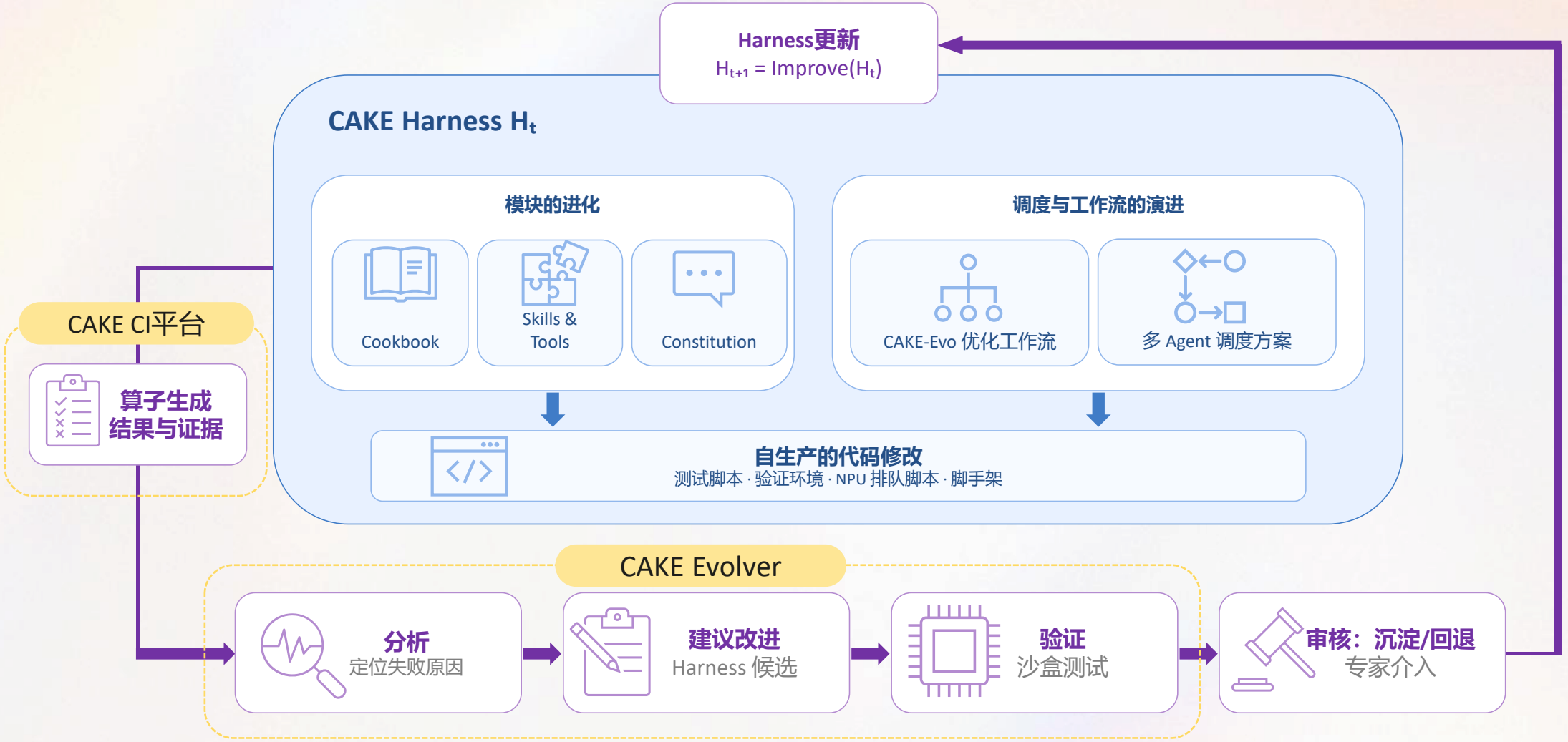
CAKE RSI 阶段 1: Handbook 自我构建验证



334 张基础知识卡片逐一构建



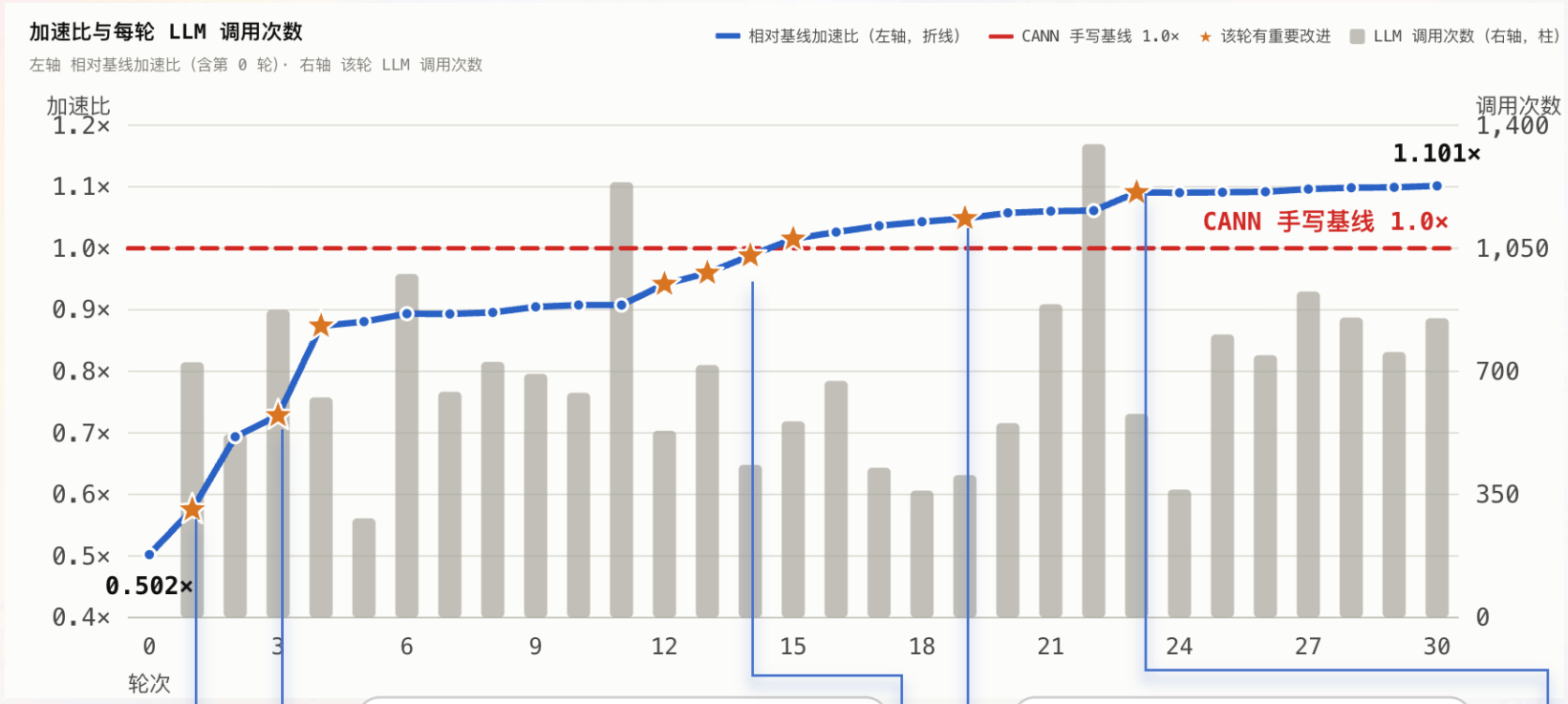
CAKE RSI 阶段 2：CAKE Harness 持续自演进



CANN AtomGit

CAKE 算子优化实战例: MLA-Prolog 算子全自动优化
典型 shape 下从 0.5x 到 1.1x, 超越原手写基线

DeepSeek MLA 复杂融合算子，基线性能 2025 年已充分优化



- RmsNorm公式:

$$RmsNorm(x)=\gamma\cdot\frac{x}{\sqrt{\frac{1}{N}\sum_{i=1}^{N}x_i^2+\varepsilon}}$$

- Query计算公式:

$$c^Q=RmsNorm(x\cdot W^{DQ})$$
$$q^C=c^Q\cdot W^{UQ}$$
$$q^N=q^C\cdot W^{UK}$$

- Query ROPE旋转位置编码:

$$C^R=ROPE(c^Q\cdot W^{QR})$$

- Key计算公式:

$$k^C=Cache(RmsNorm(x\cdot W^{DKV}))$$

- Key ROPE旋转位置编码:

$$k^R=Cache(ROPE(x\cdot W^{KR}))$$

AI核心优化点

稍减少并行度反而提升速度：AIC 28→26, AIV 56 → 52

并行掩盖：AIC 权重流掩盖AIV计算

硬件亲和：权重流按 512 字节行对齐

手写LoadData/Mmad/FixPipe取代Matmul<>模板

发现权重数据只搬运一次，绕过 L2 缓存

30+ 小时无人工干预

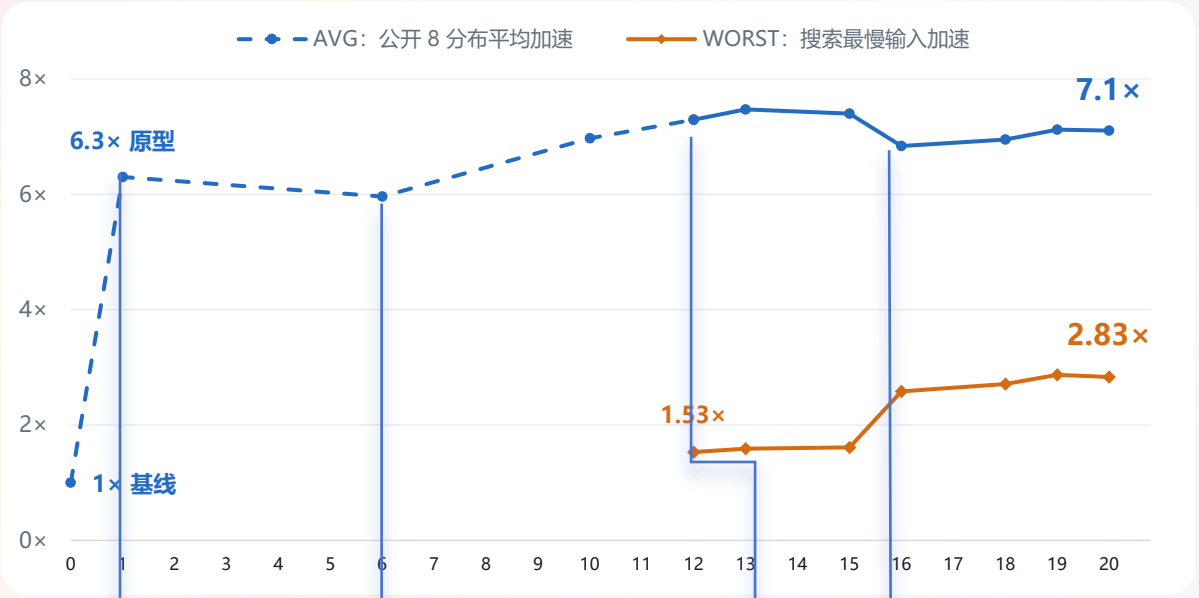
20k+ 次 LLM API 调用

最终生成 5800+ 行算子代码

CANN AtomGit

CAKE 算子优化实战例: 大规模 TopK 算子全自动优化
利用特殊 API 达成 7x 性能

N=154,256,400 K=794,880 1 D



解决正确性问题

引入成本模型：
平衡同步与保留量，
优化最差场景

R1发现原型：
蓄水池方案可达 6.3x

专家 prompt：引入对抗搜索
修改性能口径，保证最慢速度



Agent 自动识别两个技术创新点：

1. **巧妙利用 GatherMask API**
2. **实现“九分法”并行处理大量数据**

相比基线实现达成 7x 性能提升

CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit

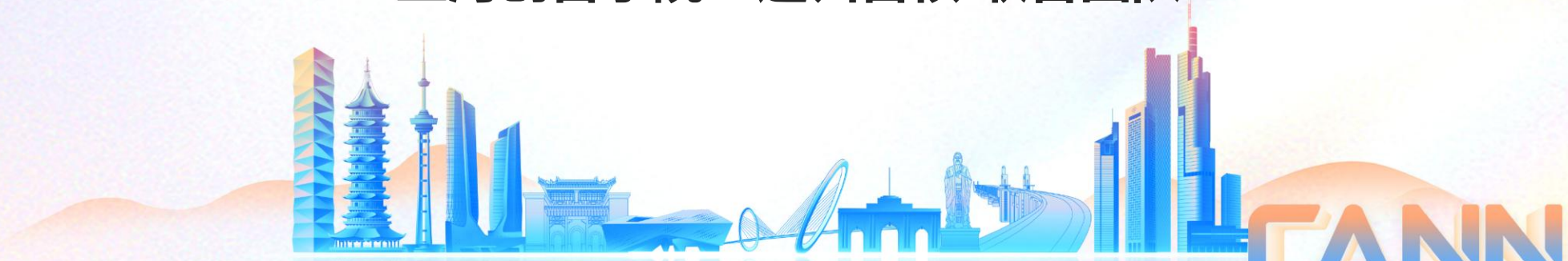


CANNBench Season 1 • 工程实践

RSI 演进之路
从算子智造到 Token 工厂

CANNBench 持续迭代背后的知识积累、真机闭环与工程实践

上海创智学院 • 递归智核 联合团队



团队介绍

上海创智学院孵化 · 跨模型、硬件与系统的工程积累

培养顶尖 AI 人才的学术机构

上海创智学院自建院起将自主智算体系作为核心科研方向，研究面向通用人工智能的计算架构与网络通信技术，构建新一代人工智能算力基础架构。

学院已建成超 20000P 智算集群，算力规模居全球高校前列；AI Infra 团队汇聚高校、芯片公司与互联网大厂的资深技术专家，在算法框架、自主卡适配与算力调度方面积累深厚。

RSI 驱动的 AI 基础设施公司

基于 RSI（递归式自我改进）技术，重构从底层算子自动生成到上层全栈 MLOps 的完整技术栈，让每一个 Token 更有价值、更省成本。

上海创智学院孵化，核心团队来自腾讯、字节、阿里、拼多多等一线大厂，覆盖从AI Infra到模型、从平台到产品的完整技术栈。

Kernaut：面向真实硬件的算子开发 Agent

递归智核 Infynai · AI for AI Infra

生成与迁移

Ascend C / Triton / CUDA

算子自动生成
已有代码适配

面向目标工具链与运行环境

验证与调优

真实硬件反馈

编译与精度对齐
真机Profiling

多轮修改与回归验证

工程交付

代码 + 报告 + 轨迹

算子代码与测试报告
可回溯优化轨迹

便于专家复核与后续回归

团队经验：模型研发 • 自主芯片适配 • 训练推理框架 • 集群交付

CANNBench：把能力放到统一标准下验证

CANNBench · official-tasks / 910C / v1.1 / solution / default

总榜名次

第 2

/ 170 个参评方案

总分 8186.17

任务覆盖

53

全部任务

公开用例通过

100%

1060 / 1060

隐藏用例通过

99.13%

4203 / 4240

隐藏侧最高分：Gather / Unique / ArgMax 等

infynai-kernaut · 快照：2026-10-15 14:33（UTC+8）；单项最高分限定到对应测试侧，可包含并列。

持续迭代，把竞争力做进系统

8 月末登顶、9 月突破 8000 分，持续迭代至封榜



8 月下旬
7000+
首轮提交

8 月下旬
7500+
继续验证与改进

9 月中旬
8000+
迈过阶段里程碑

9—10 月
持续打磨
迭代直至封榜

每轮保存可复核的实现与实验结果

基线与候选代码	正确性结果与回归结论	逐输入耗时与负结果

真机闭环：语义、精度与性能一起验证

按目标硬件组织实现，以编译、精度与真机性能反馈推进迭代

01 **规划** 语义 / 硬件约束 → 02 **候选生成** 实现与测试用例 → 03 **编译 / 精度验证** 接口 / 数值对齐 → 04 **真机计时 / Profiling** 耗时 / 瓶颈定位 → 05 **迭代精修** 修改 / 回归验证

真机结果驱动下一轮修改

语义与精度约束

shape / dtype / layout
边界输入与数值误差
与参考实现对齐

并行与存储规划

tiling / 多核划分
UB 容量与对齐约束
GM <->UB 搬运与流水

真机性能诊断

逐输入耗时
访存 / 计算利用与同步等待
根据实测修正候选

编译通过、正确性通过、性能改善分别记录；组合版本重新验证。

10,000+ 知识片段 × 50,000+ 真机验证语料

跨项目、跨平台积累的硬件知识与真实执行证据

10,000+

微架构知识片段

- 存储层级、搬运与同步约束
- Ascend C / Triton 接口与工具链经验
- 数值语义、性能机制与适用边界

50,000+

高质量真机验证语料

- 任务描述、候选代码与迭代轨迹
- 执行环境、编译与精度验证结果
- 真机耗时与性能分析记录

开发轨迹 → 验证技术结论 → 整理知识与技能 → 后续算子开发

复用前核对适用条件：芯片 / 工具链 / shape / dtype

走向 RSI：从长程迭代到优化成果回流

RSI · Recursive Self-Improvement · 递归自我改进

领先任务的探索深度

Unique **80+** 版候选探索

Level 3 • 隐藏测试侧达到榜单最高分

Gather **60+** 次线上评测

Level 2 • 隐藏测试侧达到榜单最高分

长程迭代持续保存的工程状态

版本

基线 / 候选 / 当前最佳

精度

对齐结果与回归验证

性能

逐输入耗时与回退

结论

适用条件与失败原因

经验回流：真机验证 → 知识 / 技能更新 → 下一轮优化

自举方向：优化内核 → 自身推理与实验链路 → 测量下一轮成本改善

候选与评测分别计数，包含未采用候选或失败评测；单项成绩采用 10 月 15 日快照。

落地实践：向 SGLang / vLLM 贡献高性能内核

3 个真实框架内核 · 2 个主流推理框架 · GPU / NPU 真机验证

SGLang • DeepSeek V4稀疏注意力调度算子

45.3× H200 几何平均加速

44.8× RTX 4090 几何平均加速

- O(batch) 串行 → O(log batch) 并行
- 跨硬件代次收益一致：两代 GPU 差异 <1%
- 已被上游接受合入

SGLang • 位置元数据算子

1.28× H200

2.47× 昇腾 910B

- O(bs²) 重复扫描 → 分条并行前缀和
- GPU / NPU 双平台 21/21 逐位对齐
- 同一份 Triton 算子,已提交上游 PR

vLLM • DeepSeek MoE 激活量化算子

2.57× FP8

2.68× INT8

- DeepSeek MoE 在线推理热路径
- vLLM 官方测试 650 项全部通过
- 已被上游接受合入

kernel / wrapper 级 GeoMean。调度：bs=128–32768，相对原 JIT fallback 内核；位置元数据：H200 11 个 / 910B 14 个 bs（910B 小 bs 原路径计 1.0×）；vLLM：H200、M≥4096 prefill。
各指标测试数据不同；端到端收益取决于实际负载。

Token 工厂：让局部优化服务于有效产出

把异构算力转化为稳定、高效的 Token 供给

质量约束 时延约束 可用性约束

在完整负载中测量优化价值

内核与框架

正确性与完整调用耗时

验证真实热路径占比

模型负载

吞吐 • TTFT / TPOT

明确序列长度、batch 与并发

服务产出

有效 tokens/s • 单位成本

质量、时延与可用性共同约束

最终衡量：每百万有效输出 Token 的服务成本

比较时明确模型、输入输出长度、并发与精度等条件；局部提效需经完整负载验证。

开放生态：站在巨人的肩上

CANN 的开放积累，为我们的昇腾开发与持续优化提供了重要支持

知识

cannbot-knowledge

API 语义与版本约束
源码知识与排错经验
可追溯、可复用

技能

cannbot-skills

算子开发与精度调试
性能分析等工程流程
可调用的 Agent 能力

评测

CANNBench

编译、正确性与性能
统一检验标准
持续改进的共同标尺

感谢华为 CANN 团队与社区，让工程积累成为更多开发者共同前进的基础。

让 AI 优化计算，让计算反哺 AI。

CANN AtomGit

THANKS



CANN AtomGit

茶歇交流

